

Pathogen & Vector Genomics

Practical Worksheet

Assembly & Annotation of Tick and Parasite Genomes

Duration	Content
Duration	9:30 -- 12:45 (3 h 15 min)
Part 1 --- Tick ID	Mitochondrial assembly, annotation, COX1 phylogeny (9:30 -- 11:00)
Part 2 --- Pathogen	Genome assembly, BLAST ID, guided scaffolding, polishing, QC (11:00 -- 12:45)

Sample Information	Details
Organism (tick)	Unknown tick species --- Nanopore reads
Organism (pathogen)	Unknown blood parasite --- ONT + Illumina reads
Guide reference	<i>T. orientalis</i> Shintoku (Ikeda genotype) --- GCF_000740895.1

Set up your working directory

```
# Navigate to the practical folder
cd vector_assembly/

# Confirm your data structure
ls -lh tick/
ls -lh pathogen/
ls -lh pathogen/guide/
```

■ Your data files

- tick/unknown_tick_mito_nanopore_50x.fastq --- Nanopore reads from mystery tick mitochondrion
- tick/COX1.faa --- COX1 protein sequences from reference tick species
- pathogen/SRR20016367_1.fastq.gz --- ONT reads: Unknown blood parasite
- pathogen/SRR3654148_1.fastq.gz / _2.fastq.gz --- Illumina reads: Unknown blood parasite (short-read set 1)
- pathogen/SRR3659424_1.fastq.gz / _2.fastq.gz --- Illumina reads: Unknown blood parasite (short-read set 2)
- pathogen/guide/GCA_000740895.1*_genomic.fna --- *T. orientalis* Shintoku reference genome
- pathogen/guide/genomic.gff / .gbff / .gtf --- Reference annotation files
- pathogen/guide/protein.faa --- Reference proteins (for BUSCO / annotation)

Part 1 --- Tick Identification via Mitochondrial Assembly

Duration: ~90 minutes | **Tools:** Flye, MitoZ (mitoz_env), MAFFT, IQ-TREE

Farmers have reported a mysterious disease affecting their cattle and collected some ticks across their farms. Your task is to identify the cause of this disease and the vector responsible for its transmission.

■ Learning objectives

- Perform QC on Nanopore reads and understand key metrics (read length distribution, N50, quality scores).
- Assemble a mitochondrial genome from long reads using Flye.
- Annotate the mitogenome and extract the COX1 gene.
- Construct and interpret a maximum-likelihood phylogeny to identify a species.

1.2 Genome assembly with Flye

Flye is a de novo assembler designed specifically for long, error-prone reads from Oxford Nanopore and PacBio. It builds an assembly graph from read overlaps using a repeat graph approach, which handles the repetitive regions common in mitochondrial genomes.

```
# Run Flye --- takes ~2--5 minutes for a mitogenome
flye \
--nano-hq unknown_tick_mito_nanopore_50x.fastq \
--out-dir flye_mito/ \
--genome-size 15k \
--threads 2
```

```
# While it runs, note the log output --- Flye reports iterations and graph simplifications
```

Once complete, examine the assembly output:

```
# Check assembly statistics
seqkit stats flye_mito/assembly.fasta

# Look at the assembly info file
cat flye_mito/assembly_info.txt

# How long is your assembled sequence?
grep -v '>' flye_mito/assembly.fasta | tr -d '\n' | wc -c
```

Metric	Your value	Expected
Number of contigs		1--3
Largest contig (bp)		~14,700
Is the contig circular?		Yes (circular=Y in assembly_info.txt)
Mean contig coverage		~50×

■ What does 'circular=Y' mean?

Flye detected that the contig forms a circle --- the end of the sequence overlaps with the beginning. This is the expected topology for a mitochondrial genome. A circular assembly is a strong indicator of a complete, high-quality mitogenome.

■ Discussion Questions

Q1. Your contig is ~14,700 bp. A typical tick mitogenome is ~14,700-15,000 bp. What could explain small differences in size between assemblies of the same genome?

Q2. Flye uses a 'repeat graph' rather than a de Bruijn graph. What is the key advantage of this approach for long-read data?

Q3. What would you expect to happen to assembly quality if coverage dropped from 50× to 5×?

1.3 Mitogenome annotation with MitoZ

We now need to annotate the assembled mitochondrial genome --- identifying the 13 protein-coding genes, 2 rRNAs, and 22 tRNAs that make up a typical arthropod mitogenome. We will use MitoZ, which runs in its own dedicated conda environment (mitoz_env) to avoid dependency conflicts with the main environment.

Run MitoZ annotation

```
# Activate MitoZ environment
conda deactivate
conda activate mitoz_env

# Copy the assembly and rename for clarity
cp flye_mito/assembly.fasta unknown_tick_mito_assembly.fasta

# Run MitoZ annotate
mitoz annotate \
  --thread_number 2 \
  --outprefix Unknown_tick_MIT \
  --fastafiles unknown_tick_mito_assembly.fasta \
  --genetic_code 5 \
  --clade Arthropoda \
  --fq1 unknown_tick_mito_nanopore_50x.fastq

# Note: here you may receive a visualisation error, don't worry, we can still use the output generated

# Output will be in Unknown_tick_MIT.result/
mv Unknown_tick_MIT.unknown_tick_mito_assembly.fasta.result Unknown_tick_MIT.result
ls Unknown_tick_MIT.result/
```

■ Genetic code 5 --- why?

Invertebrates (including ticks) use mitochondrial genetic code 5 (Invertebrate Mitochondrial), which differs from the standard code: TGA encodes Trp instead of Stop, and ATA encodes Met instead of Ile. Using the wrong genetic code will cause incorrect gene prediction.

When MitoZ is done, switch back to the main environment before continuing:

```
conda deactivate
conda activate vector_assembly
```

Examine the annotation output:

```
# Count annotated features
grep 'Protein coding genes totally found' Unknown_tick_MIT.result/summary.txt
grep 'tRNA genes totally found' Unknown_tick_MIT.result/summary.txt
grep 'rRNA genes totally found' Unknown_tick_MIT.result/summary.txt
```

Example

Feature type	Expected count	Your count
Protein-coding genes (CDS)	13	
tRNA genes	22	
rRNA genes (12S + 16S)	2	
Total annotated features	37	

■ Discussion Questions

Q1. A full arthropod mitogenome encodes 37 genes. If you found fewer, what might explain the missing annotations? (Hint: think about gene orientation and completeness of assembly.)

Q2. Why are mitochondrial genes often used for phylogenetics rather than nuclear genes?

1.4 Extract COX1 and build a phylogeny

COX1 (cytochrome c oxidase subunit 1) is the most widely used barcode gene for animal species identification. We will extract our COX1 sequence, add it to a reference dataset of other tick species, align them, and build a phylogenetic tree.

Extract the COX1 protein sequence

```
# Extract COX1 from MitoZ output
grep -A 1 'cox1\|COX1\|cox_1' \
  "Unknown_tick_MIT.result/Unknown_tick_MIT_unknown_tick_mito_assembly.fasta_mitoscaf.fa.gbfcds.fasta" \
  > Unknown_Cox1.fasta

# Rename the header to something clean
# Edit the header to: >Unknown_tick_sample
sed -i 's/>.*>/>Unknown_tick_sample/' Unknown_Cox1.fasta

# Append to the reference COX1 file
cat Unknown_Cox1.fasta >> COX1.fasta

# Check what species are in COX1.faa
grep '>' COX1.fasta
```

Multiple sequence alignment with MAFFT

```
# Align all COX1 sequences
mafft \
  --auto \
  --thread 2 \
  COX1.fasta > COX1_aligned.fasta

# Check alignment length
seqkit stats COX1_aligned.fasta
```

Maximum-likelihood phylogeny with IQ-TREE

```
# Build ML tree with 1000 ultrafast bootstraps
iqtree \
  -s COX1_aligned.fasta \
  -m TEST \
  -B 1000 \
  -T 2 \
  --prefix cox1_tree
```

```
# This takes ~2-5 min --- IQ-TREE will select the best substitution model
# Output: cox1_tree.treefile --- the final tree in Newick format
```

Visualise the tree

```
# Option 1 --- quick terminal preview
cat cox1_tree.treefile

# Option 2 --- upload cox1_tree.treefile to iTOL (recommended):
# https://itol.embl.de/upload.cgi
# Drag and drop the .treefile, then annotate and export
# Click on the tree >> tree structure >> 'Root the tree at midpoint'
```

■ Discussion Questions

Q1. Where does your 'unknown' tick sample cluster in the tree? Can you identify which species it belongs to based on its phylogenetic position?

Q2. What bootstrap support values do you see at key nodes? What threshold is typically considered sufficient support (e.g., $\geq 70\%$ or $\geq 95\%$)?

Q3. How does mitochondrial COX1 barcoding compare to nuclear markers for species identification? What are the limitations of a single-gene approach?

Q4. If your tick clustered with an unexpected species, what would that mean? How would you investigate further?

Part 2 --- Pathogen Genome Assembly

Duration: ~105 minutes | **Tools:** nanoq, Flye, NCBI BLAST, RagTag, Racon, BUSCO

■ Learning objectives

- Assemble a eukaryotic parasite genome from ONT reads using Flye.
- Identify the assembled organism by BLASTing against NCBI databases.
- Understand why a de novo assembly may be fragmented and how reference-guided scaffolding improves it.
- Polish an assembly using Racon to reduce Nanopore-specific sequencing errors.
- Assess assembly completeness with BUSCO.

2.2 De novo assembly with Flye

We will assemble the unknown blood parasite genome from ONT reads. This is a ~9 Mb eukaryotic genome with 4 chromosomes plus a mitochondrion and apicoplast --- significantly more complex than the tick mitogenome.

```
# Run Flye on the pathogen ONT reads
# This will take ~15-25 minutes --- so we have pre-configured this for you
# The usual cmd is:
python3 flye.py \
  --nano-hq SRR20016367_1.fastq.gz \
  --out-dir flye_pathogen/ \
  --genome-size 9m \
  --threads 2

# Note usually flye does not run using python - so just 'flye' to run in the future
```

```
# Monitor progress in the log file in another terminal:
# tail -f flye_pathogen/flye.log
```

■ This step takes time --- while Flye runs:

- Think about: what does the assembly graph of a eukaryote look like compared to a mitogenome?
- Consider: the ONT data comes from infected cattle blood --- what non-parasite sequences might be assembled?

Once complete, assess the assembly:

```
# Summary statistics
seqkit stats -a flye_pathogen/assembly.fasta

# How fragmented is the assembly?
grep -c '>' flye_pathogen/assembly.fasta

# What is the N50?
seqkit stats -a flye_pathogen/assembly.fasta | awk 'NR==2 {print "N50:", $13}'

# Sort and look at the largest contigs
seqkit sort -l -r flye_pathogen/assembly.fasta | seqkit seq --name | head -10
```

Metric	Your value	Notes
Total assembly size (Mb)		Expect ~9-15 Mb (includes host contamination)
Number of contigs		Likely tens to hundreds
N50 (bp)		Expect 100 kb -- 2 Mb range
Largest contig (bp)		
Number of circular contigs		Look in assembly_info.txt

■ Discussion Questions

Q1. Compare the number of contigs in this assembly to your tick mitogenome assembly. Why is the pathogen assembly so much more fragmented?

Q2. What does N50 tell you that the total contig count alone does not?

Q3. If total assembly size is much larger than 9 Mb, what does this suggest about the composition of the reads?

2.3 Species identification by BLAST

Before scaffolding with a reference, we want to confirm what we have assembled. We will take our largest contig and BLAST it against the NCBI nucleotide database to identify the organism. This is the 'detective' step.

Extract the largest contig

```
# Sort by length and extract the single largest contig
seqkit sort -l -r flye_pathogen/assembly.fasta \
| seqkit head -n 1 \
> largest_contig.fasta

cat largest_contig.fasta | head -2
grep -v '>' largest_contig.fasta | tr -d '\n' | wc -c
```

```
head -c 20000 largest_contig.fasta > largest_contig_20000.fasta #blast has a character limits so we subsample
```

BLAST against NCBI (web interface)

Open a browser and go to: <https://blast.ncbi.nlm.nih.gov/Blast.cgi>

- Click **Nucleotide BLAST** (blastn)
- Paste the contents of largest_contig_20000.fasta into the query box
- Under Database, select **Nucleotide collection (nr/nt)**
- Click BLAST and wait ~1--2 minutes

Rank	Subject organism / accession	% Identity / E-value / Coverage
1		
2		
3		

■ What to expect

Your top hits should be *Theileria orientalis* sequences. The closest match will likely be *T. orientalis*. You may see hits to *T. orientalis* Ikeda (Shintoku) --- the reference genome we will use for scaffolding. Ours is actually a different strain 'Chitose'

Note the % identity to Shintoku vs Chitose. Even within the same species, different genotypes can be 95--99% identical at the nucleotide level. This difference motivates the use of a guide assembly rather than just mapping directly to the reference.

■ Discussion Questions

Q1. What organism did BLAST identify? Is it the expected species and genotype?

Q3. If your largest contig BLASTed to *Bos taurus* (cattle), what would that tell you about the assembly?

2.4 Reference-guided scaffolding with RagTag

Our de novo assembly is fragmented into many contigs. RagTag uses a reference genome from a related strain to order and orient these contigs into scaffolds --- joining them with gap sequences where the order is inferred but sequence is unknown. This dramatically improves contiguity without requiring additional sequencing.

```
python3 ragtag.py scaffold \  
  GCA_000740895.1_ASM74089v1_genomic.fna \  
  flye_pathogen/assembly.fasta \  
  -o ragtag_output \  
  -t 2
```

Compare assembly statistics before and after scaffolding:

```
echo '--- Before RagTag ---'  
seqkit stats -a flye_pathogen/assembly.fasta  
  
echo '--- After RagTag ---'  
seqkit stats -a ragtag_output/ragtag.scaffold.fasta
```

Metric	Before RagTag	After RagTag
Number of sequences		
N50 (bp)		
Largest sequence (bp)		
Total assembly size (bp)		

■ Understanding RagTag output

Scaffolds containing gaps are marked with runs of 'N' nucleotides --- these are positions where RagTag placed contigs but the sequence between them is unknown.

Unplaced contigs (in ragtag.unplaced.fasta) may represent sequences absent from the Shintoku reference --- potentially Chitose-specific regions, or host contamination.

```
# Count Ns in the scaffolded assembly (= gap bases)
grep -v '>' ragtag_output/ragtag.scaffold.fasta \
| tr -d '\n' \
| grep -o 'N' \
| wc -l
```

■ Discussion Questions

Q1. RagTag inserts 'N' runs between contigs it places. How many gap bases does your scaffolded assembly contain? What is the limitation of these regions for downstream analysis?

Q2. Some contigs remained unplaced. What biological or technical reasons could explain why a contig from *T. orientalis* Chitose has no match in the Shintoku reference?

Q3. We used a different genotype (Ikeda/Shintoku) as the scaffolding reference. How might using a more closely related reference (e.g., another Chitose assembly) improve the result?

2.5 Assembly completeness with BUSCO

BUSCO (Benchmarking Universal Single-Copy Orthologs) is the standard tool for assessing genome assembly completeness. It works by searching your assembly for a curated set of genes that are expected to be present in **exactly one copy** in all members of a given lineage. The idea is simple: if your assembly is complete and accurate, you should be able to find nearly all of these conserved genes. If large numbers are missing or fragmented, something went wrong during sequencing, assembly, or both.

For apicomplexan parasites like *Theileria*, we use the **apicomplexa_odb10** database, which contains 446 conserved single-copy orthologs specific to the Apicomplexa phylum. Using a lineage-appropriate database is critical --- a generic eukaryote set would include genes absent from apicomplexans entirely, artificially inflating the "missing" score.

BUSCO reports four categories for each gene it searches for:

- **C (Complete, single-copy):** Found once --- ideal. A high C score means your assembly is complete and not redundant.
- **D (Complete, duplicated):** Found more than once --- may indicate haplotigs assembled as separate sequences, or genuine gene duplications.

- **F (Fragmented)**: Partially found --- the assembler captured the region but not fully. Often caused by low coverage or repeat-induced breaks.
- **M (Missing)**: Not found at all --- could mean the region was never sequenced, was filtered out, or is genuinely absent.

■ Example BUSCO output

Below is a representative output from a good *T. orientalis* Chitose assembly. Your results should look similar:

```
# BUSCO version is: 5.4.3
# The lineage dataset is: apicomplexa_odb10 (Creation date: 2020-09-10)
# BUSCO was run in mode: genome
```

```
-----
|Results from dataset apicomplexa_odb10      |
|-----|
|C:94.8%[S:93.1%,D:1.7%],F:1.6%,M:3.6%,n:446  |
|423      Complete BUSCOs (C)                  |
|415      Complete and single-copy BUSCOs (S)   |
|8        Complete and duplicated BUSCOs (D)    |
|7        Fragmented BUSCOs (F)                |
|16       Missing BUSCOs (M)                   |
|446      Total BUSCO groups searched           |
|-----|
```

Category	Count	Percentage
Complete single-copy (C)	415	93.1%
Complete duplicated (D)	8	1.7%
Fragmented (F)	7	1.6%
Missing (M)	16	3.6%
Total BUSCOs searched	446	100%

■ How to interpret BUSCO scores

- $C \geq 90\%$ = excellent completeness for a draft genome.
- High D (duplicated) may indicate assembly redundancy --- haplotigs assembled separately.
- High F (fragmented) suggests the assembler captured the gene region but incompletely.
- High M (missing) may indicate contaminated input data, very low coverage regions, or genuine gene loss.
- The published *T. orientalis* Chitose genome achieves ~94--96% complete BUSCOs --- compare your result!

■ Discussion Questions

Q1. How does your BUSCO completeness compare to the expected ~94--96% for a good *T. orientalis* assembly?

Q2. If you had a high 'duplicated' score, what assembly parameter could you adjust to reduce haplotig duplication?

Q3. BUSCO uses the apicomplexa_odb10 gene set. Why is it important to choose a lineage-appropriate database rather than using a generic eukaryote set?

Q4. BUSCO gives us a completeness estimate but not an accuracy estimate. What additional analysis would tell you whether the sequence itself is correct (i.e., has few errors)?

Part 3 --- Wrap-Up & Discussion

Duration: ~20 minutes | Full group debrief

3.1 Results summary

Fill in the table below with your final results from both parts of the practical:

Assembly	Key metric	Your result
Unknown tick mito (Flye)	# contigs / size / circular?	
Unknown tick mito (annotation)	# PCGs / tRNAs / rRNAs	
COX1 phylogeny	Species ID confirmed?	
Unknown pathogen (Flye raw)	# contigs / N50	
Unknown pathogen (BLAST)	Top hit organism & genotype	
Unknown pathogen (RagTag)	N50 after scaffolding	
Unknown pathogen (Racon)	% reads mapped	
Unknown pathogen (BUSCO)	% complete BUSCOs	

3.2 Group discussion questions

Discuss these broader questions with your group:

■ Discussion Questions

Q1. Comparing the two assemblies --- tick mitochondrion vs parasite nuclear genome --- what were the key differences in difficulty, and why?

Q2. We used a different genotype (Ikeda/Shintoku) as a scaffolding reference for the Chitose assembly. What are the risks of this approach? Could we introduce Ikeda-specific structural variants into our Chitose assembly?

Q3. The workflow today went: raw reads → assembly → BLAST ID → scaffold → BUSCO. For a peer-reviewed publication, what additional steps would be required before the assembly could be deposited?

Q4. We discussed EGAPx (NCBI's annotation pipeline) but did not run it today. What does EGAPx do that BUSCO does not? (Hint: one measures completeness, the other produces annotations.)

Q5. Based on your phylogenetic analysis, what tick species did you identify? How does genomic data from both the vector and pathogen together help us understand disease transmission?

Appendix --- Reference Information

A1 Key tools and their roles

Tool	Step	What it does
nanoq	QC	Fast read statistics and filtering for Nanopore FASTQ data
Flye	Assembly	De novo assembler for long error-prone reads; uses repeat graphs
MitoZ (mitoz_env)	Annotation	Full mitogenome annotation pipeline for metazoa --- runs in dedicated conda environment
MAFFT	Alignment	Multiple sequence aligner; --auto selects best strategy for input
IQ-TREE	Phylogeny	Maximum-likelihood tree inference with model selection (-m TEST)
NCBI BLAST	ID	Sequence similarity search against GenBank/nt database
RagTag	Scaffolding	Reference-guided scaffolding and patching of draft assemblies
minimap2	Mapping	Fast aligner for long reads to reference; -ax map-ont for ONT
samtools	Utilities	SAM/BAM manipulation: sort, index, flagstat
Racon	Polishing	Consensus-based assembly polishing using original reads
BUSCO	QC	Completeness assessment via conserved single-copy orthologs

A2 Glossary of key terms

Contig: A contiguous sequence assembled from overlapping reads with no gaps.

Scaffold: An ordered set of contigs separated by gaps of estimated size (represented as Ns).

N50: The length L such that 50% of the total assembly is contained in contigs/scaffolds $\geq L$. A higher N50 indicates a more contiguous assembly.

BUSCO: Benchmarking Universal Single-Copy Orthologs --- a measure of assembly completeness based on expected conserved genes.

Polishing: Correcting systematic errors in an assembly by re-examining the original read data and calling a consensus.

Genetic code 5: The invertebrate mitochondrial genetic code, used by ticks and other arthropods. Differs from the standard code at TGA (Trp, not Stop) and ATA (Met, not Ile).

COX1 / Barcode: Cytochrome c oxidase subunit 1 --- the standard animal 'DNA barcode' gene used for species identification.

Genotype (Theileria): *T. orientalis* consists of multiple genotypes (Ikeda, Chitose, Buffeli) with different geographic distributions and pathogenicities. Ikeda is the most virulent.

Apicoplast: A non-photosynthetic plastid found in apicomplexan parasites (*Theileria*, *Plasmodium*, *Toxoplasma*). Derived from secondary endosymbiosis of a red alga.

RagTag: Reference-based assembly scaffolding tool that uses nucmer alignments to order and orient contigs relative to a reference genome.

EGAPx: NCBI Eukaryotic Genome Annotation Pipeline (external). Uses RNA-seq, protein evidence, and comparative genomics to predict genes. Computationally intensive --- not run in this practical.

A3 Useful commands cheat sheet

```
# Sequence statistics
seqkit stats -a assembly.fasta # Full stats including N50
seqkit sort -l -r assembly.fasta # Sort by length (longest first)
seqkit head -n 1 assembly.fasta # Extract first sequence
grep -c '>' assembly.fasta # Count sequences

# BAM/SAM operations
samtools sort -o sorted.bam input.sam # Sort BAM
samtools index sorted.bam # Index BAM
samtools flagstat sorted.bam # Mapping statistics
samtools view -h sorted.bam > out.sam # Convert BAM to SAM

# Quick coverage check
samtools depth sorted.bam | awk '{sum+=$3; cnt++;} END {print sum/cnt}'

# Count Ns in assembly (= gap bases after scaffolding)
grep -v '>' assembly.fasta | tr -d '\n' | grep -o 'N' | wc -l

# Check gzip contents without full decompress
zcat file.fastq.gz | head -8
zcat file.fastq.gz | awk 'NR%4==1' | wc -l # Count reads
```

A4 Data provenance

File	Source / Accession	Reference
Tick mito reads	Simulated from MG450553.1 using Badread (nanopore2020 model, 50x)	Wang et al. 2019, Front. Physiol.
SRR20016367	<i>T. orientalis</i> Fish Creek (Chitose), ONT R9.4.1 MinION	Poh et al. 2022, Microorganisms
SRR3654148	<i>T. orientalis</i> Illumina PE reads	Brayton et al. 2012 / PRJNA325070
SRR3659424	<i>T. orientalis</i> Illumina PE reads	Brayton et al. 2012 / PRJNA325070
GCA_000740895.1	<i>T. orientalis</i> Shintoku (Ikeda) reference genome	Brayton et al. 2012, PLOS Pathog.
COX1.faa	COX1 proteins from reference tick species	NCBI RefSeq

End of practical worksheet.